# JavaScript Array Method - unshift()

The `unshift()` method is one of the most commonly used array methods in JavaScript. It is used to **add one or more elements to the beginning** of an array.

---

# Table of Contents



---

# What is unshift()?

## Definition

The `unshift()` method adds one or more elements to the **beginning** of an array.

It shifts all existing elements one position to the right.

It modifies the **original array** and returns the **new length** of the array.

---

# Why Use unshift()?

We use `unshift()` when we need to:

- Add an item to the beginning
- Add the newest notification first
- Add a new task at the top
- Insert priority items
- Manage queue systems

---

# Syntax

```
array.unshift(element1);

array.unshift(element1, element2, element3);
```

---

# Parameters

| Parameter | Description |
| --- | --- |
| element1 | First element to add |
| element2 | Second element (Optional) |
| element3 | Third element (Optional) |

You can add one or more elements at once.

# Return Value

`unshift()` returns the **new length** of the array.

Example

```
const fruits = ["Mango", "Orange"];

let length = fruits.unshift("Apple");

console.log(length);
```

### OUTPUT

```
3
```

# How unshift() Works

Before

```
["Mango","Orange"]
```

↓

```
unshift("Apple")
```

↓

After

```
["Apple","Mango","Orange"]
```

All existing elements move one position to the right.

---

# Example 1 (Basic)

```
const fruits = [
    "Mango",
    "Orange"
];
fruits.unshift("Apple");
console.log(fruits);
```

### OUTPUT

```
["Apple","Mango","Orange"]
```

### EXPLANATION

- `"Apple"` is added to the beginning.
- `"Mango"` moves to index `1`.
- `"Orange"` moves to index `2`.
- The original array is modified.

---

# Example 2 (Numbers)

```
const numbers = [
    20,
    30,
    40
];
numbers.unshift(10);
console.log(numbers);
```

### OUTPUT

```
[10,20,30,40]
```

# Example 3 (Multiple Elements)

```
const colors = [
    "Green",
    "Blue"
];
colors.unshift(
    "Red",
    "Black"
);
console.log(colors);
```

### OUTPUT

```
["Red","Black","Green","Blue"]
```

# Example 4 (Store Return Value)

```
const fruits = [
    "Mango",
    "Orange"
];
const newLength = fruits.unshift("Apple");
console.log(newLength);
console.log(fruits);
```

### OUTPUT

```
3
["Apple","Mango","Orange"]
```

# Example 5 (Objects)

```
const users = [
    {
        name: "Sara"
    }
];
users.unshift({
    name: "John"
});
console.log(users);
```

### OUTPUT

```
[
    { name: "John" },
    { name: "Sara" }
]
```

# Example 6 (Empty Array)

```
const numbers = [];

numbers.unshift(100);

console.log(numbers);
```

### OUTPUT

```
[100]
```

# Example 7 (Mixed Data Types)

```
const data = [
    true,
    null
];
data.unshift(
    "Rokon",
    23
);
console.log(data);
```

### OUTPUT

```
["Rokon",23,true,null]
```

# Real Project Example 1

## Notifications

```
const notifications = [

    "Payment Successful",

    "Order Shipped"

];

notifications.unshift("New Message");

console.log(notifications);
```

### OUTPUT

```
[
    "New Message",
    "Payment Successful",
    "Order Shipped"
]
```

# Real Project Example 2

## Todo List

```
const todos = [

    "Exercise",

    "Read Book"

];

todos.unshift("Study");

console.log(todos);
```

### OUTPUT

```
[
    "Study",
    "Exercise",
    "Read Book"
]
```

# Real Project Example 3

## Queue System

```
const vipQueue = [

    "Customer 2",

    "Customer 3"

];

vipQueue.unshift("VIP Customer");

console.log(vipQueue);
```

### OUTPUT

```
[
    "VIP Customer",
    "Customer 2",
    "Customer 3"
]
```

# Real Project Example 4

## Playlist

```
const playlist = [

    "Song B",

    "Song C"

];

playlist.unshift("Song A");

console.log(playlist);
```

### OUTPUT

```
[
    "Song A",
    "Song B",
    "Song C"
]
```

# Real Project Example 5

## Recent Searches

```
const searches = [
    "JavaScript",
    "React"
];
searches.unshift("Node.js");
console.log(searches);
```

### OUTPUT

```
[
    "Node.js",
    "JavaScript",
    "React"
]
```

# Best Practices

✅ Use `unshift()` only when adding elements to the beginning.

✅ Store the returned length if needed.

```
const length = array.unshift("New Item");
```

---

✅ Use meaningful variable names.

```
notifications

tasks

messages

customers
```

---

✅ Add multiple elements in one call if necessary.

```
numbers.unshift(1,2,3);
```

---

# Common Mistakes

## Mistake 1

Expecting `unshift()` to return the updated array.

Wrong

```
const result = numbers.unshift(10);

console.log(result);
```

Output

```
4
```

It returns the **new array length**, not the array.

Correct

```
numbers.unshift(10);

console.log(numbers);
```

## Mistake 2

Using `unshift()` instead of `push()`.

Remember:

```
push()
```

adds to the **end**.

```
unshift()
```

adds to the **beginning**.

---

## Mistake 3

Forgetting that indexes change.

Before

```
["Mango","Orange"]
```

After

```
unshift("Apple")
```

Result

```
["Apple","Mango","Orange"]
```

Now,

```
array[0]
```

is

```
Apple
```

---

## Mistake 4

Assuming the original array stays unchanged.

```
const arr = [2,3];

arr.unshift(1);
```

Now,

```
arr
```

becomes

```
[1,2,3]
```

---

# Interview Questions

## What does `unshift()` do?

It adds one or more elements to the **beginning** of an array.

---

## Does `unshift()` modify the original array?

✅ Yes.

---

## What does `unshift()` return?

The **new length** of the array.

---

## Can `unshift()` add multiple elements?

✅ Yes.

```
numbers.unshift(1,2,3);
```

---

## Which method adds elements to the end?

```
push()
```

---

## Which method removes the first element?

```
shift()
```

---

## What happens to existing elements after `unshift()`?

They move one position to the right.

---

# Summary Table

| Feature | Description |
| --- | --- |
| Method | `unshift()` |
| Purpose | Add element(s) to the beginning |
| Parameters | One or more elements |
| Returns | New array length |
| Changes Original Array | ✅ Yes |
| Adds To | Beginning of the array |
| Common Uses | Notifications, Queue, Todo List, Playlist |

---

# Quick Comparison

| Method | Action | Returns | Changes Original Array |
| --- | --- | --- | --- |
| `push()` | Add to end | New length | ✅ Yes |
| `pop()` | Remove from end | Removed element | ✅ Yes |
| `unshift()` | Add to beginning | New length | ✅ Yes |
| `shift()` | Remove from beginning | Removed element | ✅ Yes |

---

# Final Notes

The `unshift()` method is useful when new items should appear **first** instead of last. It is commonly used in:

- ✅ Notification systems
- ✅ News feeds

- ✅ Chat applications
- ✅ Todo lists
- ✅ Priority queues
- ✅ Search history

Mastering `unshift()` helps you efficiently manage arrays where the newest or highest-priority items belong at the beginning.